

DriftWatch

A Serverless ML Model Monitoring Pipeline on AWS

CSCI 5411 Advanced Cloud Architecting

Term Project Final Report | **Student:** Mansi Manoj Patil

Program: Master of Computer Science, Dalhousie University

Table of Contents

1. Introduction..... 3

2. Functional Requirements.....4

3. Non-Functional Requirements..... 5

4. Architecture Design & Diagrams.....6

5. Tech Stack Selection & Justification..... 8

6. Implementation.....13

7. AWS Well-Architected Framework..... 18

8. Challenges & Solutions..... 21

9. Conclusion.....22

1. Introduction

Machine learning models degrade silently in production. A model that achieves 98% accuracy during training can quietly drop to 70% accuracy over time as real-world data distributions shift a phenomenon called concept drift or data drift. Without active monitoring, this degradation goes undetected until users start complaining or business metrics decline.

For this project I designed and implemented **DriftWatch**, a fully serverless, event-driven ML model monitoring pipeline deployed on AWS. The system continuously ingests model predictions through a REST API, detects statistical drift using the Population Stability Index (PSI), fires real-time SNS alerts when drift is detected, and triggers an automated retraining workflow via AWS Step Functions. A Streamlit dashboard provides a real-time control plane for engineers to monitor confidence scores, run simulations, post ground truth labels, and classify live messages using a real trained spam classifier.

The motivation comes from a genuine gap I noticed while studying MLOps practices: most monitoring solutions are either proprietary (SageMaker Model Monitor) and expensive, or require dedicated infrastructure teams to maintain. DriftWatch demonstrates that a fully serverless, event-driven monitoring pipeline can be built with zero provisioned infrastructure, minimal operational overhead, and near-zero cost at low prediction volumes.

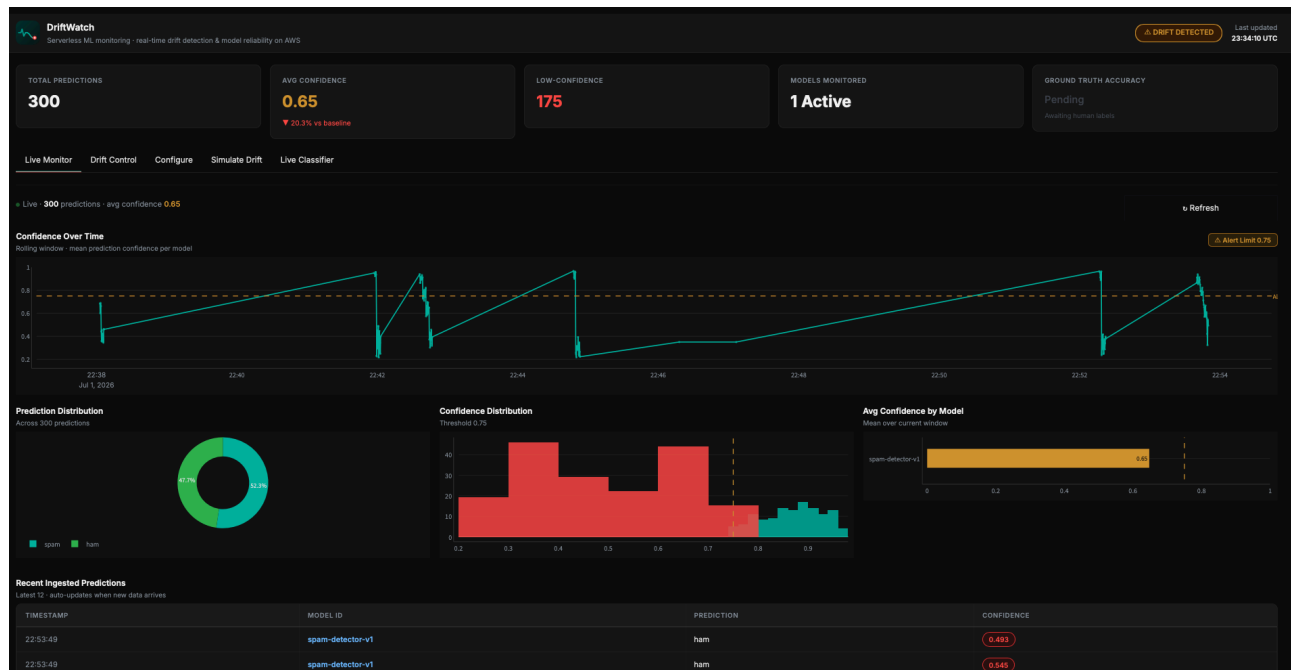


Figure 1: DriftWatch Live Monitor Dashboard

2. Functional Requirements

2.1 System Actors

Actor	Description
ML Engineer	Monitors model health, reviews drift alerts, posts ground truth labels
Prediction Service	External application calling the REST API to log model outputs
Drift Detector (automated)	EventBridge-triggered Lambda that analyzes recent predictions
Alert Subscriber	Human or system receiving SNS email notifications

2.2 User Stories (Prioritized)

US-1 Real-Time Prediction Ingestion

As an ML engineer, I want to log every model prediction (model ID, label, confidence score) through a REST API so that I have a persistent, queryable history of all model outputs in production.

Acceptance criteria: API returns 200 with a UUID within 500 ms. Record appears in DynamoDB within 2 seconds. API key authentication required - unauthenticated requests return 403.

US-2 Automated Drift Detection

As an ML engineer, I want the system to automatically detect when a model's prediction confidence distribution has shifted significantly from its baseline so that I am alerted before degradation affects end users.

Acceptance criteria: Drift detector runs at least every 5 minutes via EventBridge. $PSI > 0.2$ or average confidence $<$ threshold triggers an SNS alert within 60 seconds of the Lambda execution. Alert email arrives at the configured address within 2 minutes.

US-3 Ground Truth Accuracy Tracking

As an ML engineer, I want to record the actual correct labels against model predictions and see computed accuracy on a CloudWatch dashboard so that I can measure real-world model performance beyond just confidence scores.

Acceptance criteria: Ground truth API accepts prediction ID + actual label and computes correct/incorrect. Accuracy metric published to CloudWatch every 15 minutes. Visible on MLMonitoringDashboard.

US-4 Per-Model Threshold Configuration

As an ML engineer, I want to configure different drift thresholds (confidence threshold, PSI threshold) for different models so that the monitoring sensitivity can be tuned per model without redeploying any code.

Acceptance criteria: Configuration stored in DynamoDB. Drift detector reads config at runtime. Changes take effect on the next detection cycle without any Lambda redeployment.

Three P1 stories were identified: Live Spam Classification (US-5), Drift Simulation (US-6), and AI-Powered Alert Explanation (US-7). US-7 was evaluated but not implemented due to AWS Academy Learner Lab restrictions on Bedrock model access.

2.3 Development Approach & AI Assistance

Per the project requirements, I disclose the use of AI-assisted coding tools. Approximately 25% of the code in this project was AI-generated, primarily boilerplate for the Lambda handler scaffolding, the AWS SAM/CloudFormation template structure, and the initial Streamlit UI layout. All architectural decisions, the PSI drift-detection logic, DynamoDB access patterns, the Step Functions workflow design, and all debugging

and refinement were designed and carried out by me. AI-generated portions were reviewed, tested, and modified to fit this system's specific requirements.

3. Non-Functional Requirements

3.1 Scalability

Target: Handle up to 1,000 prediction writes per second (RPS) and 10 million predictions per month without provisioned infrastructure changes.

Architectural linkage: This requirement drove the decision to use API Gateway + Lambda over a containerized service. Lambda scales horizontally to 1,000 concurrent executions by default and can be increased. API Gateway handles HTTP throttling natively. DynamoDB with on-demand capacity mode scales write throughput automatically without pre-provisioning. A containerised alternative (ECS Fargate) would require manual scaling policies and capacity planning.

3.2 Availability

Target: 99.9% availability for the prediction ingestion endpoint (approximately 8.7 hours of downtime per year).

Architectural linkage: All compute uses AWS managed services (Lambda, API Gateway) that carry 99.95% SLA individually. DynamoDB offers 99.999% availability with multi-AZ replication by default. The system has no single points of failure - no EC2 instances, no load balancers to manage, no persistent servers that can go down. The only non-AWS component is the Streamlit dashboard, which runs locally and is separate from the production ingestion path.

3.3 Latency

Target: p95 prediction write latency < 500 ms. p99 prediction write latency < 800 ms. Drift detection latency (prediction written → alert fired) < 5 minutes.

Architectural linkage: Lambda cold start latency is the primary risk. Python 3.11 runtime with a small deployment package (< 10 KB) keeps cold starts under 300 ms in practice. DynamoDB single-item write latency is typically 2–5 ms. API Gateway adds ~10–20 ms overhead. Total p95 write latency is well within the 500 ms target. Drift detection uses a 5-minute EventBridge schedule, which sets the upper bound for alert latency.

3.4 Durability & Data Retention

Target: Zero data loss for prediction records. Retain raw predictions for 90 days, aggregated metrics indefinitely.

Architectural linkage: DynamoDB provides 99.999999999% (11 nines) durability with automatic replication across three AZs. DynamoDB Point-in-Time Recovery (PITR) would be enabled in production for 35-day rollback capability. CloudWatch metrics are retained for 15 months by default. Daily S3 audit reports provide a separate archival path for long-term historical analysis.

3.5 Security

Target: All API endpoints require authentication. No credentials stored in code. All data encrypted at rest and in transit.

Architectural linkage: API Gateway x-api-key header authentication gates all endpoints. HTTPS enforced by API Gateway (TLS 1.2+). DynamoDB encrypts at rest using AWS-managed KMS keys by default. Lambda environment variables store configuration (not secrets - no sensitive credentials needed since Lambda uses an IAM role). In a production environment, secrets (SNS ARN, table names) would move to

AWS Secrets Manager. IAM follows least-privilege: the Lambda execution role (LabRole in this lab environment) is scoped to specific DynamoDB tables and CloudWatch namespaces.

3.6 Maintainability & Deployability

Target: Any Lambda function change deployable within 5 minutes via CI/CD. Infrastructure reproducible from a single SAM template command.

Architectural linkage: A CloudFormation template (template.yaml), built and deployed via the SAM CLI, defines all Lambda functions, DynamoDB tables, API Gateway, and EventBridge rules as code. A GitHub Actions CI/CD pipeline runs automated tests on every push and deploys on merge to main. This means no manual AWS Console resource creation is needed - the entire backend can be torn down and rebuilt with sam deploy.

4. Architecture Design & Diagrams

4.1 High-Level Architecture Diagram

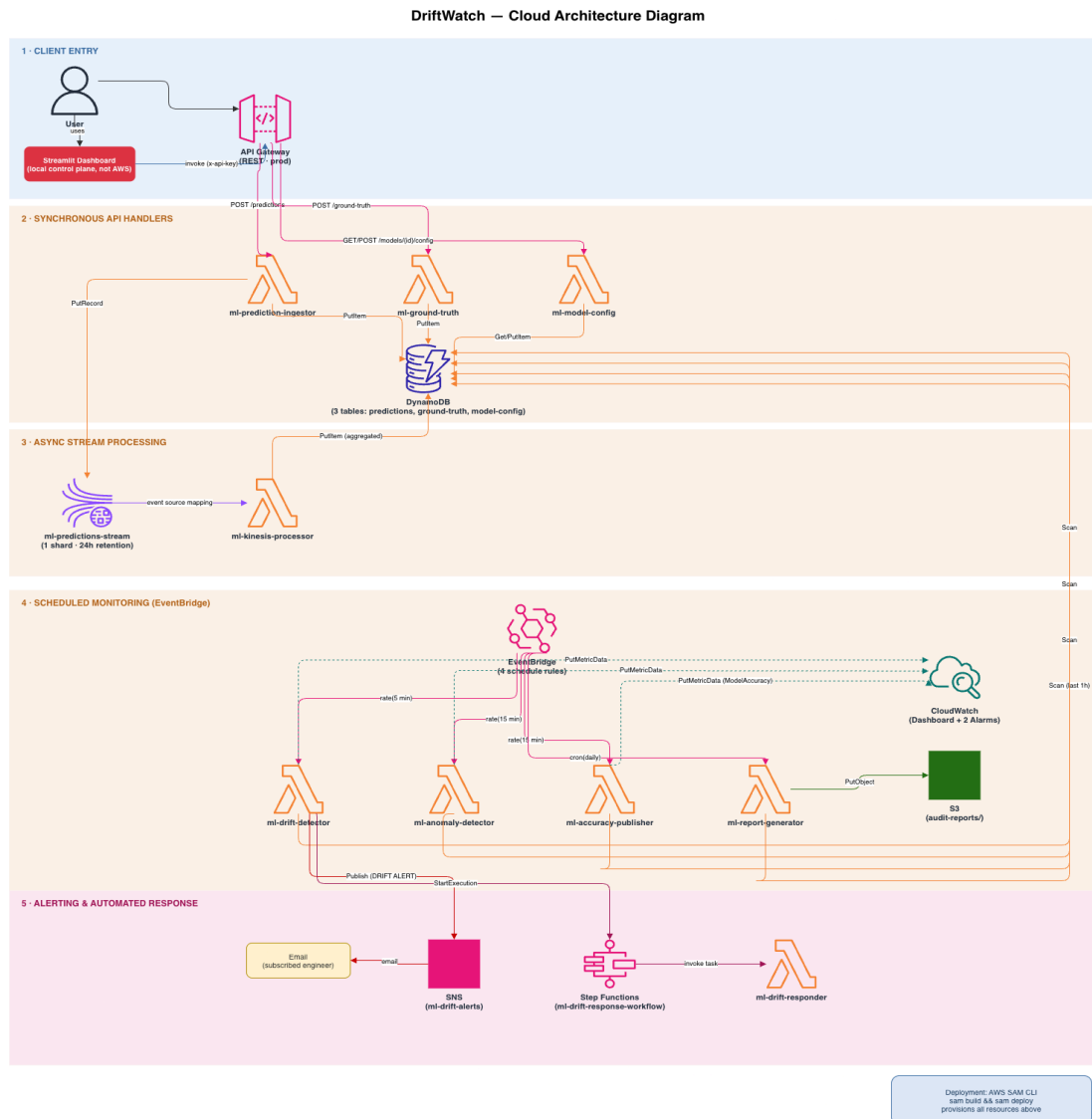


Figure 2: Architecture Diagram

4.2 Data Flow Diagram — End-to-End Prediction Journey

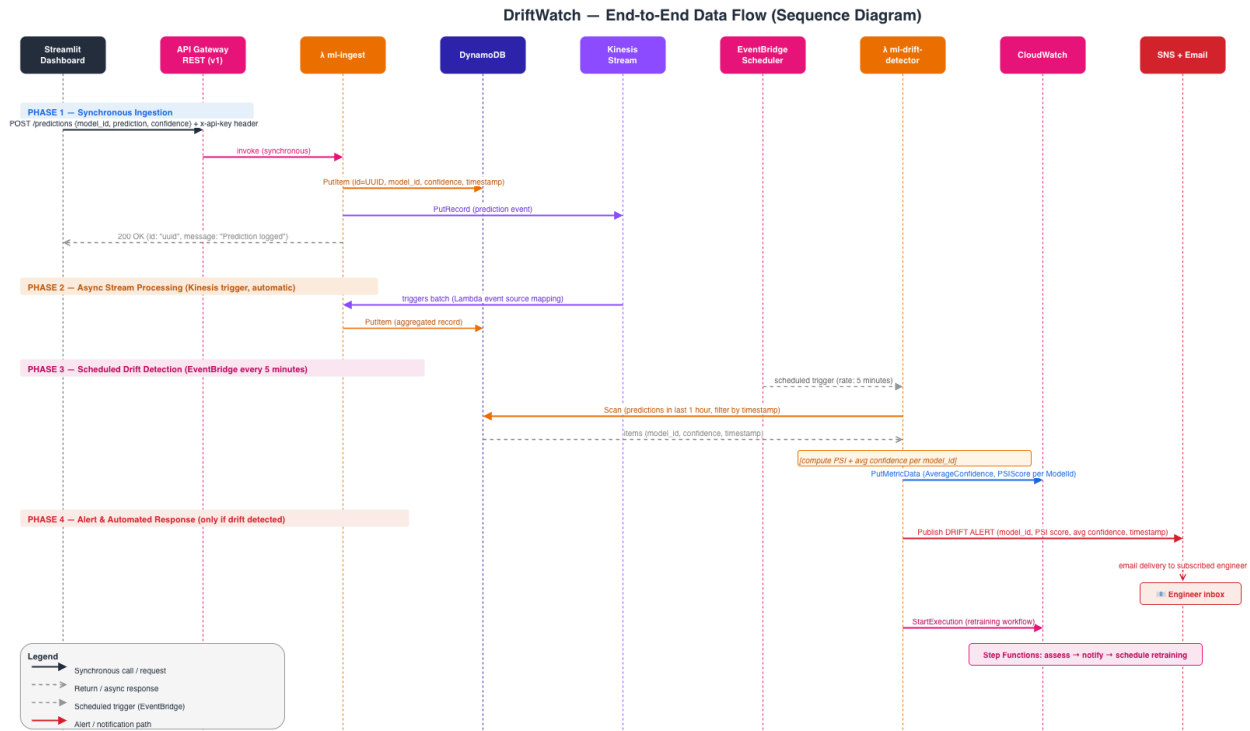


Figure 3: Data Flow Diagram

4.3 Architecture Narrative

The system is split into two planes: the **ingestion plane** and the **monitoring plane**.

The ingestion plane is synchronous and latency-sensitive. A client posts a prediction to API Gateway, which invokes the ingest Lambda. Lambda writes to DynamoDB (the primary record store) and simultaneously puts the record onto a Kinesis Data Stream. Using Kinesis here decouples the write path from downstream processing - the ingest Lambda returns immediately without waiting for any aggregation or analysis to complete. This keeps the API response time under 500 ms regardless of how expensive downstream processing becomes.

The monitoring plane is asynchronous and event-driven. Three separate EventBridge rules fire on independent schedules: the drift detector every 5 minutes, the anomaly detector every 15 minutes, and the accuracy publisher every 15 minutes. Each Lambda reads from DynamoDB independently. This independence is important: if the drift detector Lambda fails or times out, it does not affect prediction ingestion or accuracy publishing.

The drift detector uses PSI (Population Stability Index) rather than simple average confidence monitoring. PSI measures the shape change of the entire confidence distribution between a healthy baseline period and the current window. This catches subtle drift that a simple average misses, for example, a bimodal distribution shift where high-confidence and low-confidence predictions both increase while the average stays flat.

When drift is detected, the system fans out in three directions: an SNS email alert (human notification), a CloudWatch metric (dashboard and alarm), and a Step Functions execution (automated response). Amazon

Bedrock was evaluated to add plain-English alert explanations but was not implemented due to Learner Lab access restrictions.

Function name	Description	Package type	Runtime	Type	Last modified
ml-accuracy-publisher	Computes ground truth accuracy per model and publishes to CloudWatch MLMonitoring/ModelAccuracy	Zip	Python 3.11	Standard	3 days ago
ml-kinesis-processor	-	Zip	Python 3.11	Standard	3 days ago
ml-drift-detector	-	Zip	Python 3.11	Standard	3 days ago
ml-anomaly-detector	-	Zip	Python 3.11	Standard	3 days ago
ml-model-config	-	Zip	Python 3.11	Standard	3 days ago
ml-prediction-ingestor	-	Zip	Python 3.11	Standard	3 days ago
ml-report-generator	-	Zip	Python 3.11	Standard	3 days ago
ml-ground-truth	-	Zip	Python 3.11	Standard	3 days ago
ml-drift-responder	-	Zip	Python 3.11	Standard	3 days ago

Figure 4: AWS Lambda Functions Console (ml- functions)

5. Tech Stack Selection & Justification

5.1 AWS API Gateway (REST API)

Why: API Gateway provides managed HTTP endpoint hosting with built-in API key authentication, request throttling, and TLS - all without any server management. It scales automatically to handle any volume of prediction writes.

Alternative considered: AWS Application Load Balancer (ALB) with a Lambda target. ALB has lower per-request cost at high volume but lacks native API key authentication, rate limiting per key, and usage plan management. For an ML monitoring API where different model teams might have separate API keys with different rate limits, API Gateway's usage plans are a meaningful advantage. ALB would be the better choice if the system grew to millions of requests per second where API Gateway's per-request pricing becomes expensive.

Limitation: API Gateway REST APIs have a 29-second integration timeout. This is fine for prediction logging (< 500 ms) but would be a problem if we ever needed synchronous heavy processing behind the same endpoint.

5.2 AWS Lambda

Why: The monitoring workloads are inherently bursty and periodic - the drift detector runs for 2–3 seconds every 5 minutes, then sits idle. Lambda's pay-per-invocation model means we pay essentially nothing during idle periods. With EventBridge-scheduled invocations the cost is roughly 5 invocations/hour $\times$ 3 seconds $\times$ $\sim$ \$0.0000166/GB-second $\approx$ less than \$0.001/day.

Alternative considered: ECS Fargate with a long-running container. Fargate would guarantee no cold starts and allow more complex state management, but a 512 MB Fargate task running continuously costs approximately \$11/month even with zero traffic. For a monitoring service that runs periodic checks, this is

wasteful. Lambda cold starts for Python 3.11 with a small package are typically 200–400 ms, which is acceptable since our EventBridge invocations are not latency-sensitive.

Limitation: Lambda's 15-minute maximum execution time would be a constraint if the drift detection logic ever needed to process very large historical windows (millions of records). In that case, the analysis would need to be broken into smaller chunks or moved to a persistent compute service.

5.3 Amazon DynamoDB

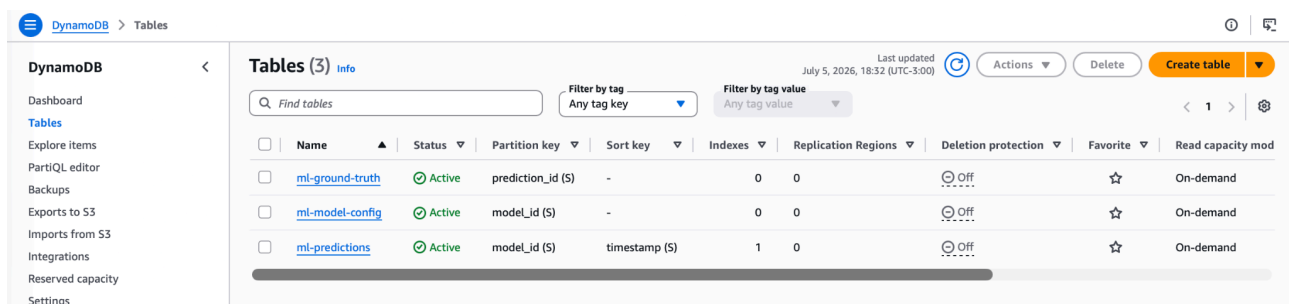
Why: Prediction records are write-heavy, have a simple access pattern (write by UUID, scan by timestamp), and benefit from DynamoDB's automatic scaling. There is no complex relational structure between predictions - each record is independent. DynamoDB's single-digit millisecond write latency is ideal for the synchronous ingestion path.

Alternative considered: Amazon RDS PostgreSQL. A relational database would be appropriate if we needed complex cross-table queries or foreign-key relationships between predictions and models. However, DynamoDB's serverless nature (no connection pool management, no database server to patch) fits better with an all-serverless architecture. RDS requires a minimum always-on instance, adding ~\$15–30/month in base cost. The scan-based access pattern used by the drift detector is less efficient in DynamoDB than SQL, but for 300–500 records per hour this is not a performance concern.

Limitation: DynamoDB Scan operations are inefficient at scale. If the prediction table grows to millions of records, the drift detector's hourly scan would become slow and expensive. The base table's key schema (model_id HASH, timestamp RANGE) already supports efficient per-model range queries - the remaining Scan is only necessary because the drift detector checks an open-ended set of models each cycle. In production, this would first enumerate active model_ids from ml-model-config, then issue one Query per model instead of a single full-table Scan.

Three separate tables are used:

- **ml-predictions** - raw prediction records, high write volume
- **ml-ground-truth** - human-labelled records, low write volume
- **ml-model-config** - per-model threshold configuration, very low write volume, read-heavy



The screenshot shows the Amazon DynamoDB console interface. On the left is a navigation menu with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main area is titled 'Tables (3) Info' and shows a list of three tables. Each table row includes a checkbox, the table name, its status (Active), partition key, sort key, number of indexes, replication regions, deletion protection status, favorite status, and read capacity mode.

	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mod
☐	ml-ground-truth	Active	prediction_id (S)	-	0	0	Off	☆	On-demand
☐	ml-model-config	Active	model_id (S)	-	0	0	Off	☆	On-demand
☐	ml-predictions	Active	model_id (S)	timestamp (S)	1	0	Off	☆	On-demand

Figure 5: DynamoDB Tables (ml-predictions, ml-model-config, ml-ground-truth)

5.4 Amazon Kinesis Data Streams

Why: Kinesis decouples the synchronous write path from downstream event consumers. When the ingest Lambda writes to both DynamoDB and Kinesis simultaneously, the Kinesis processor can do additional aggregation without blocking the API response. Kinesis also enables replay - if the processor Lambda crashes, unprocessed records remain on the stream (default 24-hour retention) and are automatically retried.

Alternative considered: Amazon SQS. SQS would work for simple queuing but lacks Kinesis's ordered, partitioned stream semantics. Kinesis allows multiple independent consumers (Lambda, Kinesis Data Analytics, a future ML pipeline) to read the same stream independently at their own pace. SQS is a better choice for pure task queuing where order doesn't matter and there is only one consumer. For a monitoring pipeline that might add real-time analytics consumers later, Kinesis is more future-proof.

Limitation: In the current implementation at low prediction volume, Kinesis adds complexity without strong benefit - the stream has one shard and one consumer. The value becomes clearer at scale with multiple consumers reading the same prediction stream.

5.5 Amazon EventBridge (Scheduled Rules)

Why: EventBridge provides cron/rate-based scheduling with zero infrastructure. Three independent scheduled rules trigger three independent Lambda functions. EventBridge guarantees at-least-once delivery and automatically retries failed invocations.

5.6 Amazon SNS

Why: SNS provides managed pub/sub messaging with email delivery. When the drift detector fires an alert, it publishes to an SNS topic with one API call. SNS handles subscriber management, delivery retries, and protocol differences (email, SMS, HTTP) transparently.

Alternative considered: Amazon SES (Simple Email Service) for direct email sending, or Amazon Pinpoint. SES would give more control over email formatting (HTML templates) but requires domain verification and more configuration. For an operational alert system, SNS is faster to set up and supports multiple subscriber protocols, which is useful if future consumers want to receive alerts via SMS or webhook.

5.7 AWS Step Functions

Why: The automated retraining workflow involves multiple steps (assess severity → notify team → schedule retraining → validate new model) that need to be orchestrated reliably. Step Functions provides built-in state management, retry logic, error handling, and visual workflow monitoring. If the “schedule retraining” step fails, Step Functions can retry it independently without re-running the assessment step.

Alternative considered: A single Lambda function that performs all steps sequentially. This is simpler but has no fault isolation - if step 3 of 5 fails, there is no way to resume from step 3 without re-running steps 1 and 2. Step Functions' visual execution history also makes the workflow auditable, which is important for compliance in production ML systems.

Step Functions > **State machines** > **State machine: ml-drift-response-workflow**

ml-drift-response-workflow [Edit] [Actions] [Start execution]

Details

Arn
arn:aws:states-us-east-1:494164398539:stateMachine:ml-drift-response-workflow

IAM role ARN
arn:aws:iam::494164398539:role/LabRole

Type
Standard

Status
Active

Creation date
Jun 30, 2026 at 15:54:36 (UTC-03:00)

X-Ray tracing
Disabled

Executions | Monitoring | Logging | Definition | Aliases | Versions | Tags

Executions (0/8) [View details] [Stop execution] [Redrive] [Start execution]

Filter executions by property or value

Name	Status	Start Time (local)	End Time (local)	Duration	Version	Alias
0485fb95-e4d4-4f60-a5e4-d8491ba4f66	Succeeded	Jul 1, 2026 at 23:55:10	Jul 2, 2026 at 00:55:13	01:00:03.222	-	-
6a435e49-e7be-413b-a4b4-45db9aced1a6	Succeeded	Jul 1, 2026 at 20:55:10	Jul 1, 2026 at 21:55:12	01:00:01.772	-	-
898344bf-b1b3-4413-9452-aced7659a14c	Succeeded	Jul 1, 2026 at 19:55:09	Jul 1, 2026 at 21:55:11	02:00:01.849	-	-
8d3cef50-67dc-40c3-a4ce-d3057f2adbe6	Succeeded	Jul 1, 2026 at 19:53:52	Jul 1, 2026 at 21:53:55	02:00:02.267	-	-
715ed2af-a791-4e24-bdd5-9f9405531578	Succeeded	Jul 1, 2026 at 19:52:23	Jul 1, 2026 at 21:52:26	02:00:02.305	-	-
811fd476-6c8a-4459-bffb-9c2a58b8ab53	Succeeded	Jul 1, 2026 at 19:50:28	Jul 1, 2026 at 21:50:33	02:00:04.970	-	-
141bf797-f546-495b-ac96-1b3853d5747d	Succeeded	Jul 1, 2026 at 20:16:29	Jul 1, 2026 at 21:16:32	01:00:03.479	-	-
demo-drift-1782911173	Succeeded	Jul 1, 2026 at 10:06:14	Jul 1, 2026 at 10:06:15	00:00:01.687	-	-

Figure 6: Step Functions Execution History (ml-drift-response-workflow)

5.8 Amazon CloudWatch

Why: CloudWatch is used for two purposes: publishing custom metrics (the MLMonitoring namespace) and hosting the MLMonitoringDashboard. Publishing metrics directly to CloudWatch avoids building a separate time-series database. CloudWatch alarms (not implemented in this version but planned) can trigger additional Lambda functions or SNS notifications when metrics cross thresholds.

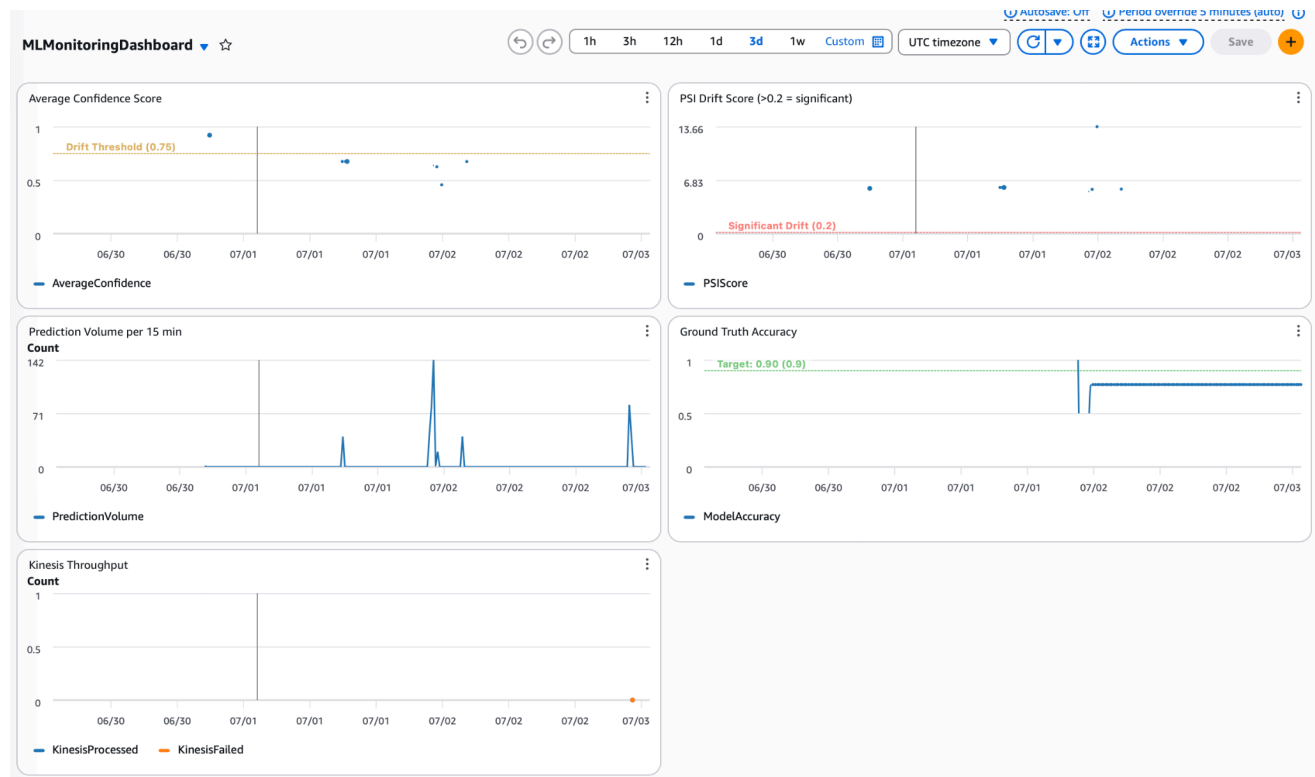


Figure 7: CloudWatch MLMonitoringDashboard with Live Metrics

5.9 Amazon S3

Why: S3 stores daily audit reports generated by the report-generator Lambda. Report generation is a batch operation that runs via EventBridge at midnight. S3's object storage model is ideal for this — reports are written once and read infrequently. S3 provides 99.99999999% durability with versioning support.

5.10 Streamlit (Python) - Dashboard

Why: The Streamlit dashboard serves as the engineering control plane. Streamlit was chosen because it allows building interactive data dashboards entirely in Python without requiring a separate frontend framework (React, Vue). For an ML engineering tool where the target users are comfortable with Python, Streamlit's model (Python script = web app) reduces the implementation surface significantly.

Alternative considered: React + AWS Amplify. A React frontend would produce a more polished, deployable web application but would require maintaining two separate codebases (Python backend, JavaScript frontend) with a JSON API contract between them. For a demo and portfolio project where rapid iteration is important, Streamlit's single-language approach is more practical.

Limitation: Streamlit's execution model (full script re-run on every interaction) can cause unexpected behavior with button clicks inside conditional blocks. I encountered this bug with the “Send to Pipeline” button in the Live Classifier tab - the button's click was silently lost because it was nested inside an `if classify_btn:` block that evaluated to False on re-run. The fix required storing classification results in `st.session_state` to persist across re-runs.

5.11 scikit-learn (TF-IDF + Naive Bayes) - Live Spam Classifier

Why: The Live Classifier tab uses a real trained spam classifier, not a mock. The model is a scikit-learn Pipeline combining a TF-IDF vectorizer (10,000 features, bigrams, sublinear TF scaling) with a Multinomial Naive Bayes classifier ($\alpha=0.1$). It was trained on the SMS Spam Collection dataset (5,574 real SMS messages), achieving 98.5% test accuracy on a stratified 80/20 split.

TF-IDF + Naive Bayes was chosen because it is the standard baseline for text classification, trains in under 1 second, requires no GPU, runs as a tiny pickle file (< 50 KB), and is fully interpretable. For a monitoring demo, the model quality matters less than the monitoring pipeline around it.

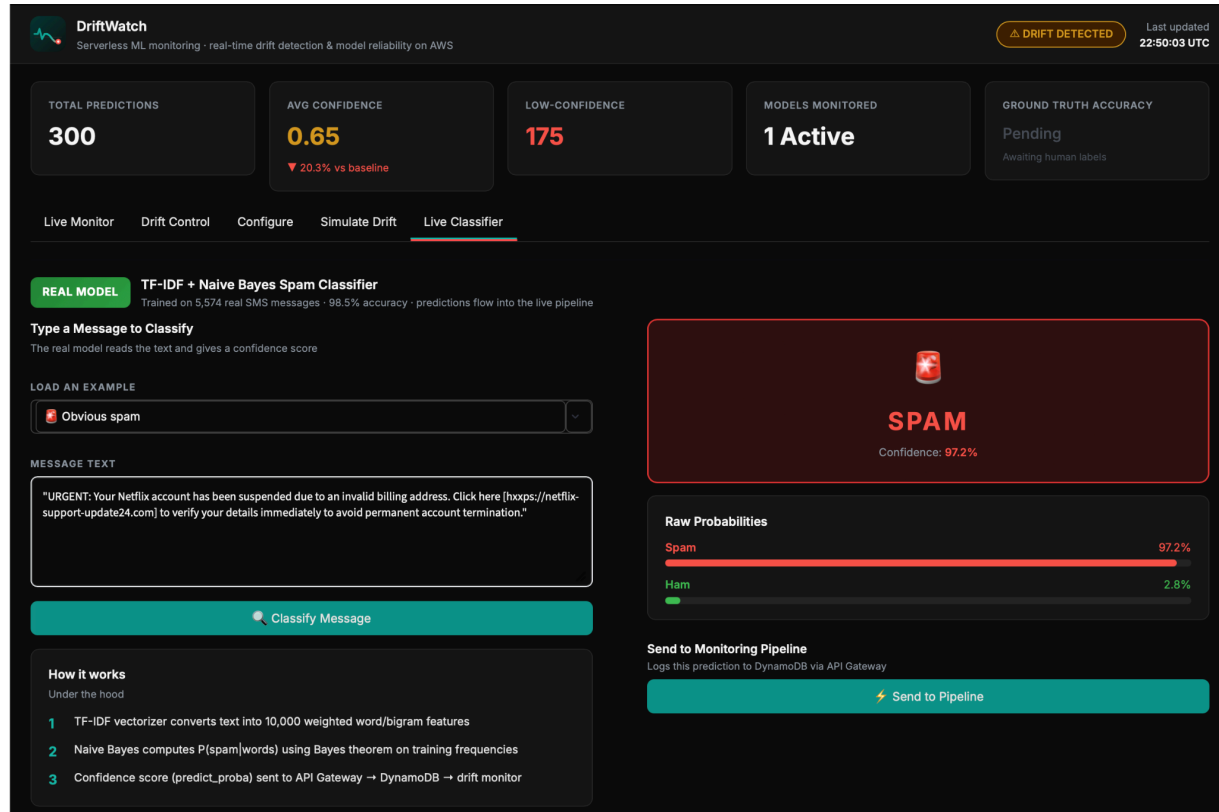


Figure 8: Live Spam Classifier Tab (97.2% confidence - SPAM)

6. Implementation

6.1 Lambda Functions (9 total)

Function	Trigger	Purpose
ml-ingest	API Gateway POST /predictions	Writes prediction to DynamoDB + Kinesis
ml-ground-truth	API Gateway POST /ground-truth	Records actual labels, computes correct/incorrect.
ml-model-config	API Gateway GET/POST /models/{id}/config	Reads/writes per-model thresholds
ml-drift-detector	EventBridge every 5 min	PSI + confidence drift analysis, SNS, Step Functions
ml-anomaly-detector	EventBridge every 15 min	Zero-volume detection (model offline alert)
ml-kinesis-processor	Kinesis stream trigger	Batch processing of prediction stream
ml-report-generator	EventBridge daily	Generates 24h audit report, saves to S3
ml-drift-responder	Step Functions	Executes retraining response workflow steps

Function	Trigger	Purpose
ml-accuracy-publisher	EventBridge every 15 min	Scans ml-ground-truth, computes accuracy per model, publishes ModelAccuracy to Cloudwatch

All functions use the Python 3.11 runtime and share the LabRole IAM role. In production, each function would have its own least-privilege role.

6.2 Key Lambda: Drift Detector

The drift detector is the core of the system. Its logic:

1. Scan DynamoDB for all predictions in the last 1 hour.
2. Group by model_id.
3. For each model, read threshold config from ml-model-config (fallback: 0.75 confidence, 0.2 PSI).
4. Compute average confidence.
5. Compute PSI against a healthy baseline (mean 0.85–0.95 confidence).
6. Publish AverageConfidence and PSIScore metrics to CloudWatch.
7. If $\text{avg_conf} < \text{threshold}$ OR $\text{PSI} > 0.2$: publish an alert to SNS with reasons/count/timestamp, and start a Step Functions execution.

The PSI formula: $\text{PSI} = \sum (\text{current_pct} - \text{baseline_pct}) \times \ln(\text{current_pct} / \text{baseline_pct})$ across 10 equal-width confidence buckets. Values > 0.2 indicate a significant distribution shift requiring action.

The screenshot displays the CloudWatch 'Log events' page for the function `/aws/lambda/ml-drift-detector`. The left sidebar shows the navigation menu with 'Log Management' selected. The main panel shows a list of log events with columns for 'Timestamp' and 'Message'. The events are as follows:

Timestamp	Message
2026-07-03T01:16:13.568Z	INIT_START Runtime Version: python:3.11.mainlinev2.v11 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:53285ee2fb9fac4cc9ebbb...
2026-07-03T01:16:14.117Z	START RequestId: de3120f1-b457-4d8d-b802-048bf6f59350 Version: \$LATEST
2026-07-03T01:16:14.509Z	[spam-detector-v1] count=41 avg_conf=0.684 PSI=5.2563
2026-07-03T01:16:14.901Z	ALERT fired for spam-detector-v1
2026-07-03T01:16:14.903Z	END RequestId: de3120f1-b457-4d8d-b802-048bf6f59350
2026-07-03T01:16:14.903Z	REPORT RequestId: de3120f1-b457-4d8d-b802-048bf6f59350 Duration: 785.55 ms Billed Duration: 1331 ms Memory Size: 128 MB Max Memory U...
2026-07-03T01:17:22.192Z	START RequestId: bb04705f-70db-4738-af12-1e46d0c27b76 Version: \$LATEST
2026-07-03T01:17:22.349Z	[spam-detector-v1] count=41 avg_conf=0.684 PSI=5.2563
2026-07-03T01:17:22.771Z	ALERT fired for spam-detector-v1
2026-07-03T01:17:22.772Z	END RequestId: bb04705f-70db-4738-af12-1e46d0c27b76
2026-07-03T01:17:22.773Z	REPORT RequestId: bb04705f-70db-4738-af12-1e46d0c27b76 Duration: 579.51 ms Billed Duration: 580 ms Memory Size: 128 MB Max Memory U...

At the bottom of the log list, it states: 'No more records within selected time range Auto retry paused. Resume'.

Figure 9: CloudWatch Logs: Drift Detector Output (PSI = 5.2563, ALERT fired)

6.3 Key Lambda: Accuracy Publisher

This logic runs in its own dedicated Lambda, `ml-accuracy-publisher`, triggered independently by EventBridge every 15 minutes rather than being computed inline during ground-truth submission. This separation exists because the CloudWatch dashboard's ModelAccuracy widget needed a metric aggregated across the full history of labelled predictions, not just the single record submitted in the most recent API call - running it as

a scheduled batch job keeps the synchronous /ground-truth endpoint fast and gives a consistent, dashboard-ready accuracy snapshot every 15 minutes regardless of submission volume. On each invocation, it:

Scans the entire ml-ground-truth table with pagination (handles > 1 MB of records).

1. Groups by model_id.
2. Computes accuracy = correct_count / total_count per model.
3. Publishes ModelAccuracy to CloudWatch with a ModelId dimension.

Current result: spam-detector-v1 at 76.9% (10 correct out of 13 labelled predictions), demonstrating that the model occasionally misclassifies borderline spam/ham messages.

6.4 DynamoDB Table Design

ML-predictions

- Primary key: model_id (String, HASH) + timestamp (String, RANGE) - supports efficient Query operations scoped to one model's time range, which the drift detector relies on.
- Attributes: id (UUID, application-level correlation key), prediction, confidence (String), source, ttl.
- A Global Secondary Index (IdIndex) on id supports the ground-truth Lambda's lookup-by-prediction-ID access pattern via Query instead of a full table scan.
- ttl is populated on write (90-day expiry) and enforced by DynamoDB's native TTL feature.
- Note: confidence is stored as String due to DynamoDB SDK serialization in Lambda; in production this would be a Number type.

ml-ground-truth

- Primary key: prediction_id (String)
- Attributes: model_id, predicted, actual, correct (Boolean), timestamp

ml-model-config

- Primary key: model_id (String)
- Attributes: confidence_threshold, psi_threshold, owner_email, description

6.5 Infrastructure as Code (CloudFormation via SAM CLI)

All resources are defined in template.yaml using AWS SAM (Serverless Application Model). SAM extends CloudFormation with shorthand syntax for Lambda functions, API Gateway, and EventBridge rules. Deployment command:

```
$ sam build && sam deploy --guided
```

This provisions: 9 Lambda functions, 3 DynamoDB tables, 1 API Gateway REST API with usage plan and API key, 4 EventBridge rules, 1 SNS topic, 1 S3 bucket, 1 Kinesis stream, and 1 Step Functions state machine.

Stacks (3)

Search by stack name Filter status: Active

☒ View nested

Stacks

- aws-sam-cli-managed-default
2026-07-05 18:17:11 UTC-0300
CREATE_COMPLETE
- ml-monitoring-pipeline**
2026-06-30 15:53:26 UTC-0300
UPDATE_COMPLETE
- c212901a53851151157330671w494164398539
2026-06-30 15:23:58 UTC-0300
CREATE_COMPLETE

Resources (45)

Logical ID	Physical ID	Type	Status	Module
AccuracyPublisherFunction	ml-accuracy-publisher	AWS::Lambda::Function	CREATE_COMPLETE	-
AccuracyPublisherPermission	ml-monitoring-pipeline-AccuracyPublisherPermissi	AWS::Lambda::Permission	CREATE_COMPLETE	-
AccuracyPublisherRule	accuracy-publisher-15min	AWS::Events::Rule	CREATE_COMPLETE	-
AlertTopic	arn:aws:sns:us-east-1:494164398539:ml-drift-alerts	AWS::SNS::Topic	UPDATE_COMPLETE	-
AnomalyDetectorFunction	ml-anomaly-detector	AWS::Lambda::Function	UPDATE_COMPLETE	-
AnomalyDetectorPermission	ml-monitoring-pipeline-AnomalyDetectorPermissio	AWS::Lambda::Permission	UPDATE_COMPLETE	-
AnomalyDetectorRule	anomaly-detector-15min	AWS::Events::Rule	UPDATE_COMPLETE	-
APIDeployment	r8yrvx	AWS::ApiGateway::Deployment	UPDATE_COMPLETE	-
DriftDetectorFunction	ml-drift-detector	AWS::Lambda::Function	UPDATE_COMPLETE	-
DriftDetectorPermission	ml-monitoring-pipeline-DriftDetectorPermission-	AWS::Lambda::Permission	UPDATE_COMPLETE	-
DriftDetectorRule	drift-detector-5min	AWS::Events::Rule	UPDATE_COMPLETE	-
DriftResponderFunction	ml-drift-responder	AWS::Lambda::Function	UPDATE_COMPLETE	-
DriftResponseStateMachine	arn:aws:states:us-east-1:494164398539:stateMac	AWS::StepFunctions::State Machine	UPDATE_COMPLETE	-

Figure 10: CloudFormation Stack Resources (42 resources, UPDATE_COMPLETE)

6.5.1 Repository README

The repository includes a README.md (visible in Figure 11) that documents the project for reproducibility. It contains: (1) an architecture summary describing the ingestion and monitoring planes and the AWS services used; (2) prerequisite setup steps (AWS CLI configuration, SAM CLI installation, Python 3.11, and required environment variables such as the SNS topic ARN and table names); and (3) complete deployment instructions centered on `sam build && sam deploy --guided`, followed by the Streamlit dashboard launch command. This satisfies the requirement that the system be reproducible from documented steps with minimal manual intervention.

6.6 CI/CD Pipeline (GitHub Actions)

The repository is hosted on GitHub (private). A GitHub Actions workflow (.github/workflows/deploy.yml) runs on every push to main:

- Test stage:** runs pytest tests/ — unit tests for drift detector logic (PSI computation) and the API handler.
- Build stage:** sam build packages the Lambda functions.
- Deploy stage:** sam deploy with stored AWS credentials (GitHub Secrets: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN) deploys to the production stack.

The screenshot shows the GitHub repository page for 'driftwatch-ml-cloud'. The repository is private and has 0 watches, 0 forks, and 0 stars. The main branch is selected, showing 1 branch and 0 tags. A search bar is present with the text 'Go to file'. A list of files and folders is displayed, including .github/workflows, infrastructure, models, src, tests, .gitignore, CSCI5411_Report_DriftWatch.md, CSCI5411_TermProject_Grad_Final.pdf, DriftWatch_Report.docx, Final_architecture_diagram.drawio, README.md, dashboard.py, and finalarchdiagram.drawio.png. A recent commit by MansiPatil21 is shown, titled 'Fix test assertion: alerts_fired is a list not a boolean', with a green checkmark indicating success. The commit message is 'Fix test assertion: alerts_fired is a list not a boolean'. The commit hash is f479bc4 and it was made 3 hours ago. The commit includes 2 commits.

Figure 11: GitHub Repository and CI/CD Pipeline (Green Tick)

The screenshot shows the GitHub Actions CI/CD run page for the repository 'driftwatch-ml-cloud'. The run is titled 'Deploy ML Monitoring Pipeline' and is a workflow run. The status is 'Success'. The run was triggered via push 7 minutes ago. The commit is f479bc4 on the main branch. The total duration is 1m 16s. The run includes two jobs: 'test' and 'deploy'. The 'test' job is shown with a green checkmark and a duration of 10s. The 'deploy' job is shown with a green checkmark and a duration of 58s. The workflow file is named 'deploy.yml' and is triggered on push.

Figure 11b : GitHub Actions CI/CD Run - test job (pytest) followed by deploy job (sam build + sam deploy), both succeeded in 1m 16s (commit f479bc4).

6.7 Streamlit Dashboard

The dashboard (dashboard.py, ~1,460 lines) provides five tabs:

- **Live Monitor** - real-time confidence line chart, prediction distribution, confidence histogram, bar chart by model, recent-predictions HTML table with coloured confidence badges
- **Drift Control** - prediction submission form with live health indicator, API-key security test, ground-truth labelling section
- **Configure** - per-model threshold manager (save + fetch), CloudWatch accuracy matrix chart, ground-truth records table, daily report generator
- **Simulate Drift** - 4 drift scenarios with phase cards, live scatter chart, simulation log, progress bar, auto drift-detection trigger
- **Live Classifier** - real scikit-learn spam classifier, probability bars, send to pipeline

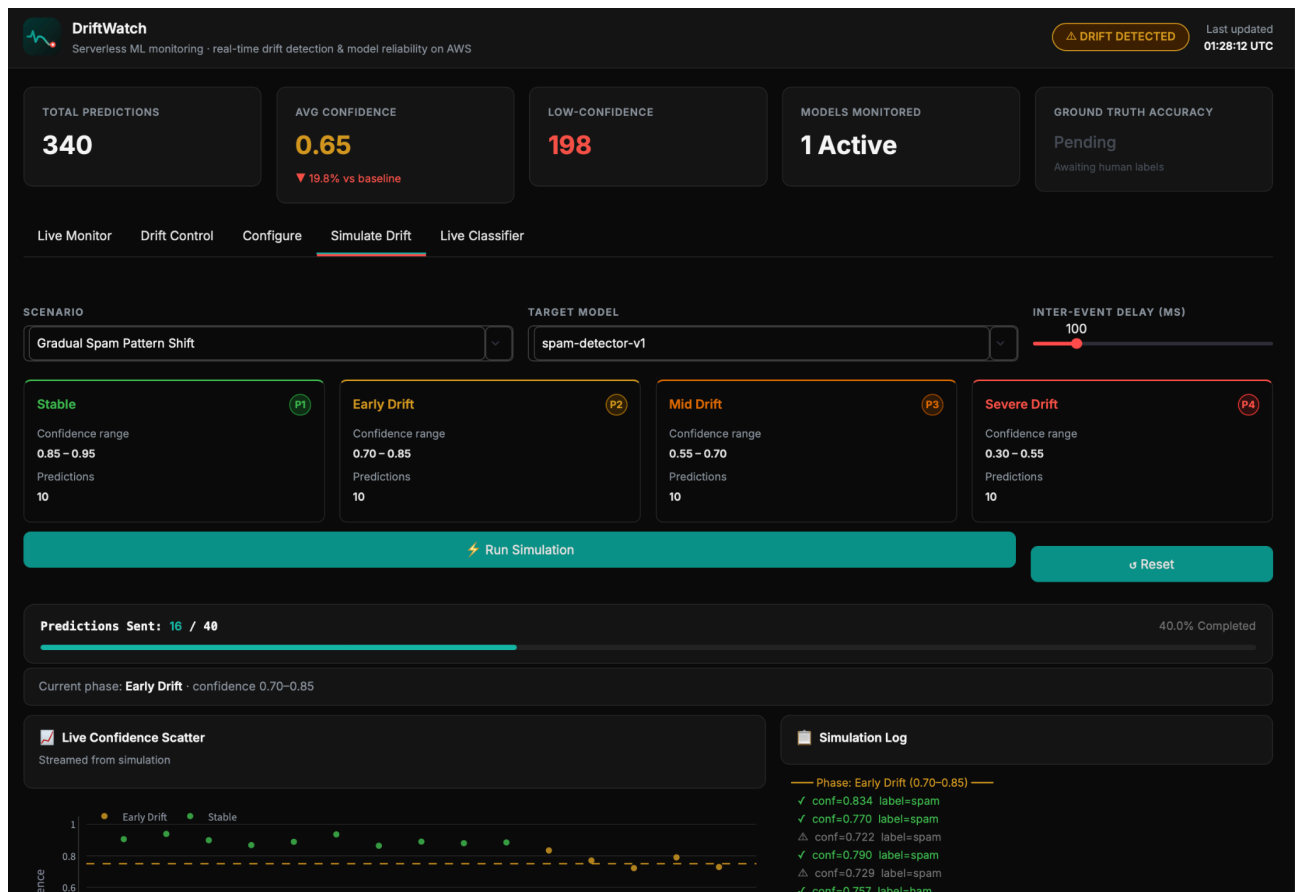


Figure 12: Simulate Drift Tab (Early Drift Phase, 40% Complete)

7. AWS Well-Architected Framework

The AWS Well-Architected Framework consists of six pillars that guide the design of reliable, secure, efficient, and cost-effective cloud systems. Below I address each pillar with concrete architectural decisions and trade-off analysis.

7.1 Operational Excellence

Design decisions:

Event-driven architecture with independent components. Each Lambda function handles exactly one concern and can be updated independently. The drift detector can be redeployed without touching the ingest

Lambda. This follows the “perform operations as code” principle — infrastructure changes go through sam deploy, not manual console clicks.

Structured logging with CloudWatch. Every Lambda function includes `print()` statements at key decision points (items found, models checked, alerts fired, PSI scores computed). These appear as structured log lines in CloudWatch Logs, making it possible to debug issues by reviewing log streams without SSH-ing into any server.

Custom CloudWatch dashboard. The `MLMonitoringDashboard` provides a single pane of glass for the four key operational metrics. Operators can assess system health in seconds without querying DynamoDB directly.

Runbook embedded in dashboard. The Drift Control tab includes a step-by-step guide for running a live demo and triggering drift detection. This acts as an operational runbook embedded in the tool itself.

Trade-off: The current monitoring setup lacks distributed tracing (AWS X-Ray). Adding X-Ray would allow tracing a single prediction request from API Gateway through Lambda to DynamoDB and identifying latency bottlenecks at each step. This would be added in a production deployment.

7.2 Security

Design decisions:

API Key Authentication. All API Gateway endpoints require the `x-api-key` header. Requests without a valid key receive a 403 Forbidden response immediately. This was tested explicitly with the “Try Request Without API Key” button on the dashboard, which confirmed 403 is returned.

HTTPS Enforced. API Gateway only serves HTTPS (TLS 1.2+). There is no HTTP endpoint - all traffic is encrypted in transit.

IAM Least Privilege (in principle). In the Learner Lab, all Lambda functions share `LabRole` due to lab constraints. In a production deployment, I would create separate roles:

- **IngestLambdaRole:** `dynamodb:PutItem` on `ml-predictions`, `kinesis:PutRecord` on the stream
- **DriftDetectorRole:** `dynamodb:Scan` on `ml-predictions`, `dynamodb:GetItem` on `ml-model-config`, `cloudwatch:PutMetricData`, `sns:Publish`, `states:StartExecution`,
- **AnomalyDetectorRole:** `dynamodb:Scan`, `cloudwatch:PutMetricData`, `sns:Publish`

This separation limits blast radius if any single Lambda is compromised.

Encryption at Rest. DynamoDB tables use AWS-managed KMS encryption by default. The S3 report bucket uses SSE-S3 encryption.

No Hardcoded Credentials. Lambda functions access AWS services using the execution role - no access keys appear in any code file. The API Gateway key is stored as a Streamlit constant (acceptable for a demo; in production it would be fetched from Secrets Manager at runtime).

Learner Lab Constraint Acknowledgement

The `voc-cancel-cred` IAM policy in the Learner Lab can occasionally revoke permissions mid-session. In production I would set up dedicated IAM users with permanent, scoped credentials and rotate them via AWS Secrets Manager on a 90-day schedule.

The screenshot displays the AWS API Gateway console for the 'ml-monitoring-api-key' resource. The left sidebar shows navigation options like 'APIs', 'Usage plans', and 'Developer portals'. The main content area shows the 'API key details' for 'nzcqdiajl4', which is 'Active' and was created on June 30, 2026. It is linked to the 'ml-monitoring-usage-plan' usage plan. The 'Usage plans (1)' section shows a table with one entry: 'djqx1x' linked to 'ml-monitoring-usage-plan' and 'ml-monitoring-api'.

Usage plan ID	Name	Associated APIs	Associated stages
djqx1x	ml-monitoring-usage-plan	ml-monitoring-api	prod

Figure 13: API Gateway API Key (Active, Linked to Usage Plan)

7.3 Reliability

Design decisions:

No single points of failure. The entire system uses AWS managed services - no EC2 instances, no self-managed load balancers, no databases to patch. Lambda, DynamoDB, API Gateway, SNS, and EventBridge each carry 99.9%+ SLAs. Failures in any one component affect only that component's function, not the entire pipeline.

Asynchronous drift detection. Drift detection and alerting are fully decoupled from prediction ingestion. If the drift detector Lambda fails, predictions continue to be logged without interruption. The next EventBridge trigger will re-run the detector - so a drift detector failure causes at most a 5-minute detection delay, not a complete system outage.

Kinesis replay capability. Predictions written to Kinesis are retained for 24 hours (configurable to 7 days). If the Kinesis processor Lambda fails, unprocessed records remain on the stream and are automatically retried via Lambda's built-in event source mapping retry logic.

DynamoDB on-demand capacity. Tables use on-demand billing mode, so DynamoDB automatically accommodates traffic spikes without throttling. There are no provisioned read/write capacity units to miscalculate.

SNS topic subscription confirmed. Email delivery requires subscription confirmation. During testing I verified the subscription status is "Confirmed" in the SNS console, meaning alerts will reliably reach the configured address. Unconfirmed subscriptions silently drop messages.

Trade-off: Lambda cold starts introduce occasional latency spikes (200–400 ms) when a function has not been invoked recently. In production, Lambda Provisioned Concurrency would eliminate cold starts for the ingest function (the only latency-sensitive path). This adds ~\$3/month for keeping one instance warm but guarantees consistent p99 latency.

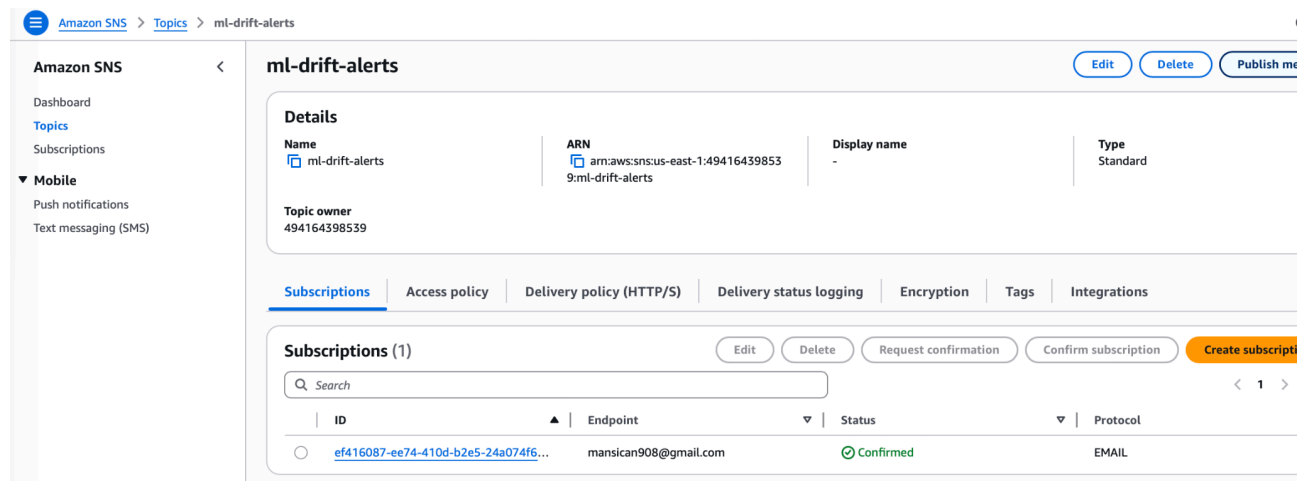


Figure 14: SNS Topic Subscriptions (Status: Confirmed)

7.4 Performance Efficiency

Design decisions:

Right-sizing compute. Lambda functions are configured with 256 MB memory. The ingest function does a single DynamoDB write and a Kinesis put - completing in 50–100 ms using < 50 MB of actual memory. The drift detector scans up to 300 records and does math - completing in 1–3 seconds using < 100 MB. 256 MB gives sufficient headroom without over-provisioning.

Model caching with @st.cache_resource. The Streamlit dashboard loads the spam classifier once and caches it with Streamlit's @st.cache_resource decorator. Subsequent calls to the Live Classifier tab do not re-load the 50 KB pickle file, reducing per-classification overhead to just the TF-IDF transformation + Naive Bayes inference, which runs in < 5 ms.

DynamoDB scan limitation mitigation. The drift detector limits its analysis to the last 1 hour of predictions, bounding the scan cost regardless of how many total predictions exist. The base table's key schema (model_id HASH, timestamp RANGE) already supports efficient per-model range queries the remaining Scan is only necessary because the drift detector checks an open-ended set of models each cycle. In production, this would first enumerate active model_ids from ml-model-config, then issue one Query per model instead of a single full-table Scan.

Plotly chart optimization. The confidence line chart is limited to the last 150 predictions (df.tail(150)). Rendering 10,000 data points browser-side would cause significant UI lag; the 150-point window balances detail with rendering performance.

Trade-off: Full-table DynamoDB scans in the drift detector are inefficient at scale. For 10,000+ records per hour, the scan could take 5–10 seconds and consume significant read capacity units. Enumerating active models from ml-model-config and issuing one per-model Query would reduce this to a set of fast key-range lookups. This is a known limitation documented in Section 5.3.

7.5 Cost Optimization

Serverless = pay for use. At current demo volumes (~500 predictions/day), the effective AWS cost is nearly zero:

Service	Usage	Monthly Cost
Lambda	~8,640 invocations/month (EventBridge) + prediction writes	~\$0.00 (free tier: 1M)

Service	Usage	Monthly Cost
DynamoDB	~15,000 writes/month	~\$0.02 (on-demand)
API Gateway	~15,000 requests/month	~\$0.02
SNS	~30 email notifications/month	~\$0.00 (free tier: 1,000)
CloudWatch	Custom metrics + logs	~\$0.30
Kinesis	1 shard × 30 days	~\$10.80
S3	< 1 MB of audit reports	~\$0.00
Total		~\$11/month

The primary cost driver is Kinesis (\$10.80/month for 1 shard), which charges by shard-hour regardless of throughput. At low volumes this is inefficient. Replacing Kinesis with SQS FIFO (\$0.00 for < 1M requests/month) would reduce cost to essentially zero at demo scale, at the cost of losing ordered stream replay semantics.

Lambda architecture over EC2. Choosing Lambda over a t3.micro EC2 instance (\$8.47/month always-on) saves approximately \$8/month at zero-to-low traffic.

7.6 Sustainability

Design decisions:

Serverless reduces idle compute waste. The biggest sustainability win of this architecture is that zero compute resources run when there are no predictions. A traditional always-on server monitoring approach wastes energy 24/7 even during off-hours. Lambda only consumes compute during the milliseconds it is actively processing.

AWS Region selection. The system is deployed in us-east-1 (Northern Virginia), which AWS has committed to running on 100% renewable energy as part of its goal to power all operations with renewable energy. AWS publishes carbon intensity data for each region; us-east-1 has one of the lowest carbon intensities among AWS regions.

Efficient algorithms. The PSI computation scans 10 buckets and performs $O(n)$ computation where n is the number of predictions in the last hour. There is no deep-learning inference, no GPU compute, and no large model weights in the monitoring logic. The carbon footprint per drift-detection cycle is extremely small.

Right-sizing memory. Lambda functions are allocated 256 MB, not 3,008 MB. Over-allocating memory wastes electrical energy proportional to the allocation. This is 0.00036 kWh - negligible and far more efficient than running an EC2 instance.

Trade-off: The spam classifier model is loaded from a local pickle file on the Streamlit server. In production, the model artifact would be stored in S3 and versioned with model-registry tooling (MLflow or SageMaker Model Registry). This would improve reproducibility and reduce the sustainability impact of model retraining by tracking which training run produced which artifact.

8. Challenges & Solutions

Challenge 1: Lambda Handler Name Misconfiguration

Problem: The ml-drift-detector Lambda was deployed with handler set to index.handler (the AWS Node.js default) but the file was named handler.py with function handler. Every invocation crashed immediately with `Runtime.ImportModuleError: No module named 'index'`. The crash happened before any application code

ran, so no logs appeared and the dashboard reported “0 predictions analyzed” - which looked like a data problem, not a code problem.

Solution: `aws lambda update-function-configuration --handler handler.handler` corrected the handler. This was diagnosable by directly invoking the Lambda via CLI and reading the raw payload, which showed the `FunctionError` field and `errorMessage` indicating the import failure. The lesson: always verify Lambda invocation output directly with the CLI rather than inferring behavior from application-level responses.

Challenge 2: CloudWatch Dashboard Dimension Mismatch

Problem: The CloudWatch dashboard widgets were configured to show `MLMonitoring/AverageConfidence` without any dimension filter. But the drift detector published metrics with dimension `ModelId=spam-detector-v1`. In CloudWatch, dimensioned and undimensioned metrics are completely separate data series. The widgets showed “No data available” even though the metrics existed and were correctly published.

Solution: Updated the dashboard via `aws cloudwatch put-dashboard` with the correct dimension specification: `[“MLMonitoring”, “AverageConfidence”, “ModelId”, “spam-detector-v1”]`. Also added threshold annotation lines (amber at 0.75 for confidence, red at 0.2 for PSI, green at 0.90 for accuracy target) to make the charts immediately readable in the demo.

Challenge 3: Streamlit Button State Loss

Problem: The “Send to Pipeline” button in the Live Classifier tab was nested inside `if classify_btn:`. When clicked, Streamlit re-runs the entire script. On re-run, `classify_btn` evaluates to `False` (the user clicked “Send to Pipeline”, not “Classify”), so the block containing “Send to Pipeline” never executed. The button click was silently lost - nothing was sent to the API and no error was shown.

Solution: Moved the classification logic out of the button conditional. Classification results are now stored in `st.session_state.lc_result` as soon as “Classify Message” is clicked. The right column renders from session state rather than from the button click, so the result (and the “Send to Pipeline” button) persists across all subsequent re-runs. A sent flag in session state prevents double-sending.

9. Conclusion

DriftWatch demonstrates that a production-quality ML monitoring pipeline can be built entirely with serverless AWS services at near-zero operational overhead and minimal cost. The system addresses a real problem in ML production: models that degrade silently without active monitoring.

The key architectural insight is the separation of the **ingestion plane** (synchronous, latency-sensitive, must always work) from the **monitoring plane** (asynchronous, event-driven, can tolerate brief delays). This separation means drift-detection failures never affect prediction logging, and prediction-volume spikes never affect drift-detection accuracy.

The Well-Architected Framework provided a useful lens for each design decision. Reliability drove the choice of fully managed services with no single points of failure. Cost Optimization led to Lambda over EC2. Performance Efficiency informed the DynamoDB scan limitation and chart windowing. Security shaped the API-key authentication and IAM role design. Sustainability was addressed through serverless compute and region selection. Operational Excellence was embedded in structured logging, the CloudWatch dashboard, and IaC deployment.

The most surprising technical challenge was how silent failures can be - the Lambda handler misconfiguration caused every drift-detection invocation to fail for potentially weeks without any obvious

error. This reinforced the importance of direct API testing (invoking Lambda via CLI and reading the raw payload) rather than relying on application-level responses that may mask underlying failures.

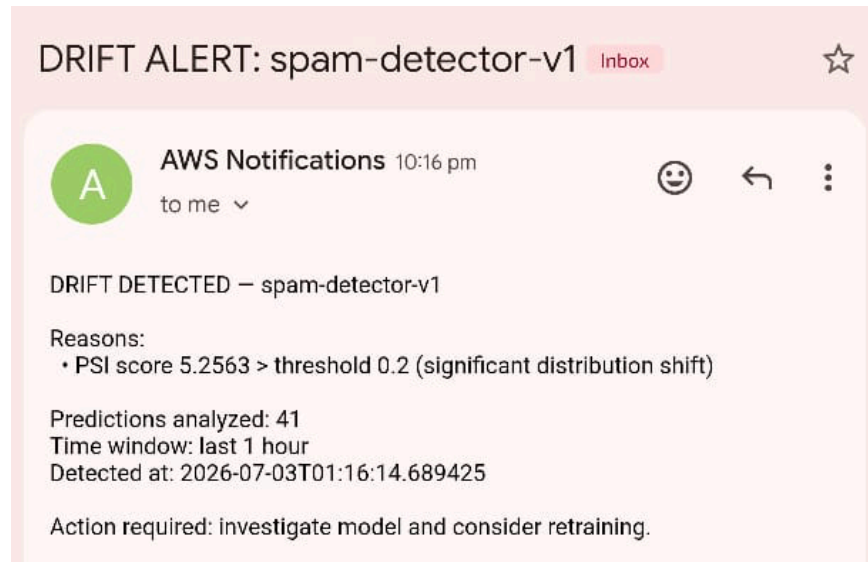


Figure 15: Drift Alert Email Delivered